

Hope Artificial Intelligence

Classification Assignment

Problem Statement or Requirement:

A requirement from the Hospital, Management asked us to create a predictive model which will predict the **Chronic Kidney Disease (CKD)** based on the several parameters. The Client has provided the dataset of the same.

1) Identify your problem statement:

- **STAGE-1(Domain Selection)** – Machine Learning as mention my client we need to predict if the person is CKD or Not by using CKD Dataset.
- **STAGE-2(Learning Selection)**- Supervised Learning Requirement from the client Input are clear and Output present.
- **STAGE-3**- Classification because output variable are Categorical values yes/no.

2) Tell basic info about the dataset (Total number of rows, columns)

- Rows = 399
- Columns = 28

3) Mention the pre-processing method if you're doing any (like converting string to number – nominal data)

- Converted Categorical columns into numerical by using One hot encoding because these are Nominal Data's.
- And Used StandardScaler in independent variable dataset because standardized the values 0 to 1.

4) Develop a good model with good evaluation metric. You can use any machine learning algorithm; you can create many models. Finally, you have to come up with final model.

- **SVC:**
 - 1) Accuracy = 98.5%
 - 2) Precision = 98.1%
 - 3) Recall = 98.7%
 - 4) F1 Score = 98.4%
 - 5) ROC_AUC Score = 99.9%
- **RandomForestClassifier:**
 - 1)Accuracy = 98.5%
 - 2) Precision = 98.2%

- 3) Recall = 98.8%
- 4) F1 Score = 98.5%
- 5) ROC_AUC Score = 99.8%

*** DecisionTreeClassifier:**

- 1) Accuracy = 97
- 2) Precision = 96.9
- 3) Recall = 96.9
- 4) F1 Score = 97
- 5) ROC_AUC Score = 96.8

- **KNN:**

- 1) Accuracy = 94.0%
- 2) Precision = 93.5%
- 3) Recall = 95.0%
- 4) F1 Score = 94.0%
- 5) ROC_AUC Score = 98.5

- **LogisticRegression:**

- 1) Accuracy = 97.0%
- 2) Precision = 96.6%
- 3) Recall = 97.1%
- 4) F1 Score = 97.0
- 5) ROC_AUC Score = 99.8

- 5) All the research values of each algorithm should be documented. (You can make tabulation or screenshot of the results.)

- **SVC Algorithm:**

Best_parameter:- f1_macro for the best parameter{'C': 10, 'gamma': 'auto', 'kernel': 'sigmoid'}: 0.9850064683509054

params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_score
{'C': 10, 'gamma': 'auto', 'kernel': 'rbf'}	1.000000	0.981217	1.0	1.000000	1.000000	0.996243	0.007513	3
{'C': 10, 'gamma': 'auto', 'kernel': 'linear'}	0.963284	0.962573	1.0	0.981233	0.981014	0.977621	0.013837	25
{'C': 10, 'gamma': 'auto', 'kernel': 'poly'}	1.000000	0.981217	1.0	0.961755	1.000000	0.988594	0.015265	21
{'C': 10, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.981569	1.000000	1.0	1.000000	1.000000	0.996314	0.007372	1
gamma: 'scale', 'kernel': 'rbf'	1.000000	0.981217	1.0	1.000000	1.000000	0.996243	0.007513	3
{'C': 10, 'gamma': 'scale', 'kernel': 'linear'}	0.963284	0.962573	1.0	0.981233	0.981014	0.977621	0.013837	25
{'C': 10, 'gamma': 'scale', 'kernel': 'poly'}	1.000000	0.981217	1.0	0.961755	1.000000	0.988594	0.015265	21
{'C': 10, 'gamma': 'scale', 'kernel': 'sigmoid'}	0.981569	1.000000	1.0	1.000000	1.000000	0.996314	0.007372	1
{'C': 100, 'gamma': 'auto', 'kernel': 'rbf'}	1.000000	0.981217	1.0	1.000000	1.000000	0.996243	0.007513	3
gamma: 'auto', 'kernel': 'linear'}	0.963284	0.962573	1.0	0.981233	0.981014	0.977621	0.013837	25
{'C': 100, 'gamma': 'auto', 'kernel': 'poly'}	1.000000	0.981217	1.0	0.981014	0.981233	0.988693	0.009233	19
{'C': 100, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.981569	0.962573	1.0	0.981233	0.981014	0.981278	0.011837	23
{'C': 100, 'gamma': 'scale', 'kernel': 'rbf'}	1.000000	0.981217	1.0	1.000000	1.000000	0.996243	0.007513	3
{'C': 100, 'gamma': 'scale', 'kernel': 'linear'}	0.963284	0.962573	1.0	0.981233	0.981014	0.977621	0.013837	25

	'gamma': 'scale', 'kernel': 'poly'}	1.000000	0.981217	1.0	0.981014	0.981233	0.988693	0.009233	19
	{'C': 100, 'gamma': 'scale', 'kernel': 'sigmoid'}	0.981569	0.962573	1.0	0.981233	0.981014	0.981278	0.011837	23
	{'C': 1000, 'gamma': 'auto', 'kernel': 'rbf'}	1.000000	0.981217	1.0	1.000000	1.000000	0.996243	0.007513	3
	{'C': 1000, 'gamma': 'auto', 'kernel': 'linear'}	0.963284	0.962573	1.0	0.981233	0.981014	0.977621	0.013837	25
	{'C': 1000, 'gamma': 'auto', 'kernel': 'poly'}	1.000000	0.962573	1.0	0.981014	1.000000	0.988717	0.014998	13
	C': 1000, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.981569	0.962573	1.0	0.981233	0.962264	0.977528	0.014083	35
	C': 1000, 'gamma': 'scale', 'kernel': 'rbf'}	1.000000	0.981217	1.0	1.000000	1.000000	0.996243	0.007513	3
	C': 1000, 'gamma': 'scale', 'kernel': 'linear'}	0.963284	0.962573	1.0	0.981233	0.981014	0.977621	0.013837	25
	C': 1000, 'gamma': 'scale', 'kernel': 'poly'}	1.000000	0.962573	1.0	0.981014	1.000000	0.988717	0.014998	13
i	{'C': 1000, 'gamma': 'scale', 'kernel': 'sigmoid'}	0.981569	0.962573	1.0	0.981233	0.962264	0.977528	0.014083	35
f	{'C': 2000, 'gamma': 'auto', 'kernel': 'rbf'}	1.000000	0.981217	1.0	1.000000	1.000000	0.996243	0.007513	3
r	{'C': 2000, 'gamma': 'auto', 'kernel': 'linear'}	0.963284	0.962573	1.0	0.981233	0.981014	0.977621	0.013837	25
/	{'C': 2000, 'gamma': 'auto', 'kernel': 'poly'}	1.000000	0.962573	1.0	0.981014	1.000000	0.988717	0.014998	13

{'C': 2000, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.981569	0.962573	1.0	0.981233	0.962264	0.977528	0.014083	35
{'C': 2000, 'gamma': 'scale', 'kernel': 'rbf'}	1.000000	0.981217	1.0	1.000000	1.000000	0.996243	0.007513	3
{'C': 2000, 'gamma': 'scale', 'kernel': 'linear'}	0.963284	0.962573	1.0	0.981233	0.981014	0.977621	0.013837	25
{'C': 2000, 'gamma': 'scale', 'kernel': 'poly'}	1.000000	0.962573	1.0	0.981014	1.000000	0.988717	0.014998	13
{'C': 3000, 'gamma': 'auto', 'kernel': 'rbf'}	1.000000	0.981217	1.0	1.000000	1.000000	0.996243	0.007513	3
{'C': 3000, 'gamma': 'auto', 'kernel': 'linear'}	0.963284	0.962573	1.0	0.981233	0.981014	0.977621	0.013837	25
{'C': 3000, 'gamma': 'auto', 'kernel': 'poly'}	1.000000	0.962573	1.0	0.981014	1.000000	0.988717	0.014998	13
{'C': 3000, 'gamma': 'auto', 'kernel': 'sigmoid'}	0.981569	0.962573	1.0	0.981233	0.962264	0.977528	0.014083	35
{'kernel': 'sigmoid'}	0.981569	0.962573	1.0	0.981233	0.962264	0.977528	0.014083	35
{'C': 3000, 'gamma': 'scale', 'kernel': 'rbf'}	1.000000	0.981217	1.0	1.000000	1.000000	0.996243	0.007513	3
{'C': 3000, 'gamma': 'scale', 'kernel': 'linear'}	0.963284	0.962573	1.0	0.981233	0.981014	0.977621	0.013837	25
{'C': 3000, 'gamma': 'scale', 'kernel': 'poly'}	1.000000	0.962573	1.0	0.981014	1.000000	0.988717	0.014998	13
{'C': 3000, 'gamma': 'scale', 'kernel': 'sigmoid'}	0.981569	0.962573	1.0	0.981233	0.962264	0.977528	0.014083	35

- **RandomForestClassifier:**

Best_parameter:- f1_macro with best parameter:{'criterion': 'gini', 'max_features': 'sqrt', 'n_estimators': 80} 0.9699248120300752

params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_score
{'criterion': 'gini', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	17
{'criterion': 'gini', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	17
{'criterion': 'gini', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	17
{'criterion': 'gini', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	17
{'criterion': 'gini', 'max_features': 'sqrt', ...}	0.981569	0.981217	0.981014	0.961755	0.962264	0.973564	0.009437	16
{'criterion': 'gini', 'max_features': 'sqrt', ...}	1.000000	1.000000	1.000000	0.961755	0.981014	0.988554	0.015284	4
{'criterion': 'gini', 'max_features': 'sqrt', ...}	1.000000	1.000000	1.000000	0.981014	0.981014	0.992406	0.009301	1
{'criterion': 'gini', 'max_features': 'sqrt', ...}	1.000000	1.000000	1.000000	0.961755	0.981014	0.988554	0.015284	4
{'criterion': 'gini', 'max_features': 'log2', ...}	1.000000	0.981217	1.000000	0.961755	0.981014	0.984797	0.014285	14
{'criterion': 'gini', 'max_features': 'log2', ...}	1.000000	1.000000	1.000000	0.961755	0.981014	0.988554	0.015284	4
{'criterion': 'gini', 'max_features': 'log2', ...}	1.000000	1.000000	1.000000	0.961755	0.981014	0.988554	0.015284	4
{'criterion': 'gini', 'max_features': 'log2', ...}	1.000000	1.000000	1.000000	0.961755	0.981014	0.988554	0.015284	4
{'criterion': 'entropy', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	17
{'criterion': 'entropy', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	17
{'criterion': 'entropy', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	17
{'criterion': 'entropy', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	17
{'criterion': 'entropy', 'max_features': 'sqrt', ...}	1.000000	1.000000	1.000000	0.961755	0.981014	0.988554	0.015284	4

{'criterion': 'entropy', 'max_features': 'sqrt'...	0.981569	1.000000	1.000000	0.961755	0.981014	0.984868	0.014268	13
{'criterion': 'entropy', 'max_features': 'sqrt'...	1.000000	1.000000	1.000000	0.961755	0.981014	0.988554	0.015284	4
{'criterion': 'entropy', 'max_features': 'sqrt'...	1.000000	1.000000	1.000000	0.961755	0.981014	0.988554	0.015284	4
{'criterion': 'entropy', 'max_features': 'log2'...	1.000000	0.962573	1.000000	0.961755	0.981014	0.981068	0.016923	15
{'criterion': 'entropy', 'max_features': 'log2'...	1.000000	1.000000	1.000000	0.981014	0.981014	0.992406	0.009301	1
{'criterion': 'entropy', 'max_features': 'log2'...	1.000000	1.000000	1.000000	0.961755	0.981014	0.988554	0.015284	4
{'criterion': 'entropy', 'max_features': 'log2'...	1.000000	1.000000	1.000000	0.981014	0.981014	0.992406	0.009301	1

- Logistic Regression:**

Best_parameter:- f1_macro with best parameter{'penalty': 'l2', 'solver': 'saga':
0.9774780806716137

params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_score
{'penalty': 'l2', 'solver': 'newton- cg'}	0.981569	0.981217	1.000000	1.000000	1.000000	0.992557	0.009116	2
{'penalty': 'l2', 'solver': 'lbfgs'}	0.981569	0.981217	1.000000	1.000000	1.000000	0.992557	0.009116	2
{'penalty': 'l2', 'solver': 'liblinear'}	0.963284	1.000000	0.981233	0.981233	0.981233	0.981397	0.011612	4
{'penalty': 'l2', 'solver': 'saga'}	0.963284	1.000000	1.000000	1.000000	1.000000	0.992657	0.014687	1

- DecisionTreeClassifier:**

- Best_parameter:- The f1_macro with best_params{'criterion': 'entropy', 'max_features': 'sqrt', 'splitter': 'random'}: 0.9699248120300752

	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_score
	{'criterion': 'gini', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	9
	{'criterion': 'gini', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	9
	{'criterion': 'gini', 'max_features': 'sqrt', ...}	1.000000	0.962573	0.962636	0.943041	0.925272	0.958704	0.024895	4
	{'criterion': 'gini', 'max_features': 'sqrt', ...}	0.981569	0.981217	1.000000	0.906166	0.944161	0.962623	0.033556	3
	{'criterion': 'gini', 'max_features': 'log2', ...}	0.945100	0.888286	0.906935	0.961755	0.924528	0.925321	0.026187	8
	random {'criterion': 'gini', 'max_features': 'log2', ...}	0.945100	1.000000	0.981233	0.962636	0.962264	0.970247	0.018761	2
	best {'criterion': 'entropy', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	9
	random {'criterion': 'entropy', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	9
	best {'criterion': 'entropy', 'max_features': 'sqrt', ...}	0.906891	0.962573	1.000000	0.943041	0.961755	0.954852	0.030283	5
	random {'criterion': 'entropy', 'max_features': 'sqrt', ...}	0.981569	0.962573	1.000000	0.962264	0.981233	0.977528	0.014083	1
	best {'criterion': 'entropy', 'max_features': 'log2', ...}	0.981569	0.962573	0.981014	0.885265	0.943699	0.950824	0.035619	7
	random {'criterion': 'gini', 'max_features': 'log2', ...}	0.945100	1.000000	0.981233	0.962636	0.962264	0.970247	0.018761	2
	best {'criterion': 'entropy', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	9
	random {'criterion': 'entropy', 'max_features': 'auto', ...}	NaN	NaN	NaN	NaN	NaN	NaN	NaN	9
	best {'criterion': 'entropy', 'max_features': 'sqrt', ...}	0.906891	0.962573	1.000000	0.943041	0.961755	0.954852	0.030283	5
	random {'criterion': 'entropy', 'max_features': 'sqrt', ...}	0.981569	0.962573	1.000000	0.962264	0.981233	0.977528	0.014083	1
	best {'criterion': 'entropy', 'max_features': 'log2', ...}	0.981569	0.962573	0.981014	0.885265	0.943699	0.950824	0.035619	7
	random {'criterion': 'entropy', 'max_features': 'log2', ...}	0.963284	0.923652	0.981233	0.944161	0.961755	0.954817	0.019506	6

- **KNN Algorithm:**
- Best_parameter: The f1_macro with best_params{'metric': 'minkowski', 'n_neighbors': 4, 'p': 2}: 0.9403772589368158

	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_score
	{'metric': 'minkowski', 'n_neighbors': 4, 'p': 2}	0.981569	0.981217	1.000000	0.944161	0.944161	0.970222	0.022337	1
	{'metric': 'minkowski', 'n_neighbors': 6, 'p': 2}	0.945100	0.981217	0.981233	0.962636	0.962636	0.966564	0.013575	4
	{'metric': 'minkowski', 'n_neighbors': 8, 'p': 2}	0.908877	0.962573	0.962636	0.944161	0.962636	0.948177	0.020909	10
	{'metric': 'sqeuclidean', 'n_neighbors': 4, 'p': 2}	0.981569	0.981217	1.000000	0.944161	0.944161	0.970222	0.022337	1
	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_score
	{'metric': 'minkowski', 'n_neighbors': 4, 'p': 2}	0.981569	0.981217	1.000000	0.944161	0.944161	0.970222	0.022337	1
	{'metric': 'minkowski', 'n_neighbors': 6, 'p': 2}	0.945100	0.981217	0.981233	0.962636	0.962636	0.966564	0.013575	4
	{'metric': 'minkowski', 'n_neighbors': 8, 'p': 2}	0.908877	0.962573	0.962636	0.944161	0.962636	0.948177	0.020909	10
	{'metric': 'sqeuclidean', 'n_neighbors': 4, 'p': 2}	0.981569	0.981217	1.000000	0.944161	0.944161	0.970222	0.022337	1
2	{'metric': 'sqeuclidean', 'n_neighbors': 6, 'p': 2}	0.945100	0.981217	0.981233	0.962636	0.962636	0.966564	0.013575	4
2	{'metric': 'sqeuclidean', 'n_neighbors': 8, 'p': 2}	0.908877	0.962573	0.962636	0.944161	0.962636	0.948177	0.020909	10
2	{'metric': 'euclidean', 'n_neighbors': 4, 'p': 2}	0.981569	0.981217	1.000000	0.944161	0.944161	0.970222	0.022337	1
2	{'metric': 'euclidean', 'n_neighbors': 6, 'p': 2}	0.945100	0.981217	0.981233	0.962636	0.962636	0.966564	0.013575	4
2	{'metric': 'euclidean', 'n_neighbors': 8, 'p': 2}	0.908877	0.962573	0.962636	0.944161	0.962636	0.948177	0.020909	10

2	{'metric': 'sqeuclidean', 'n_neighbors': 6, 'p':...	0.945100	0.981217	0.981233	0.962636	0.962636	0.966564	0.013575	4
2	{'metric': 'sqeuclidean', 'n_neighbors': 8, 'p':...	0.908877	0.962573	0.962636	0.944161	0.962636	0.948177	0.020909	10
2	{'metric': 'euclidean', 'n_neighbors': 4, 'p': 2}	0.981569	0.981217	1.000000	0.944161	0.944161	0.970222	0.022337	1
2	{'metric': 'euclidean', 'n_neighbors': 6, 'p': 2}	0.945100	0.981217	0.981233	0.962636	0.962636	0.966564	0.013575	4
2	{'metric': 'euclidean', 'n_neighbors': 8, 'p': 2}	0.908877	0.962573	0.962636	0.944161	0.962636	0.948177	0.020909	10
	{'metric': 'hamming', 'n_neighbors': 4, 'p': 2}	0.926978	0.888515	0.981233	0.944161	0.944161	0.937010	0.030034	15
	{'metric': 'hamming', 'n_neighbors': 6, 'p': 2}	0.963284	0.925524	0.981233	0.944161	0.943699	0.951580	0.019039	9
	{'metric': 'hamming', 'n_neighbors': 8, 'p': 2}	0.926978	0.944023	1.000000	0.944161	0.943699	0.951772	0.024995	7
	{'metric': 'russellrao', 'n_neighbors': 4, 'p':...	0.200200	0.206772	0.189203	0.189203	0.189203	0.194916	0.007299	16
	{'metric': 'russellrao', 'n_neighbors': 6, 'p':...	0.200200	0.206772	0.189203	0.189203	0.189203	0.194916	0.007299	16
	{'metric': 'hamming', 'n_neighbors': 4, 'p': 2}	0.926978	0.888515	0.981233	0.944161	0.944161	0.937010	0.030034	15
	{'metric': 'hamming', 'n_neighbors': 6, 'p': 2}	0.963284	0.925524	0.981233	0.944161	0.943699	0.951580	0.019039	9
	{'metric': 'hamming', 'n_neighbors': 8, 'p': 2}	0.926978	0.944023	1.000000	0.944161	0.943699	0.951772	0.024995	7
	{'metric': 'russellrao', 'n_neighbors': 4, 'p':...	0.200200	0.206772	0.189203	0.189203	0.189203	0.194916	0.007299	16
	{'metric': 'russellrao', 'n_neighbors': 6, 'p':...	0.200200	0.206772	0.189203	0.189203	0.189203	0.194916	0.007299	16

{'metric': 'russellrao', 'n_neighbors': 8, 'p': ...	0.200200	0.206772	0.189203	0.189203	0.189203	0.194916	0.007299	16
{'metric': 'canberra', 'n_neighbors': 4, 'p': 2}	0.926978	0.925524	0.962636	0.962636	0.944161	0.944387	0.016279	13
{'metric': 'canberra', 'n_neighbors': 6, 'p': 2}	0.926978	0.925524	0.962636	0.962636	0.925272	0.940609	0.017994	14
{'metric': 'canberra', 'n_neighbors': 8, 'p': 2}	0.926978	0.925524	1.000000	0.962636	0.943699	0.951768	0.027615	8
{'metric': 'russellrao', 'n_neighbors': 8, 'p': ...	0.200200	0.206772	0.189203	0.189203	0.189203	0.194916	0.007299	16
{'metric': 'canberra', 'n_neighbors': 4, 'p': 2}	0.926978	0.925524	0.962636	0.962636	0.944161	0.944387	0.016279	13
{'metric': 'canberra', 'n_neighbors': 6, 'p': 2}	0.926978	0.925524	0.962636	0.962636	0.925272	0.940609	0.017994	14
{'metric': 'canberra', 'n_neighbors': 8, 'p': 2}	0.926978	0.925524	1.000000	0.962636	0.943699	0.951768	0.027615	8

6) Mention your final model, justify why u have chosen the same

- I evaluated multiple models including Decision Tree, Random Forest, KNN, and SVM using metrics such as **Accuracy, Precision, Recall, F1 Score, and ROC-AUC**.
- Among them, **SVM and Random Forest gave the best performance**. For example,
- **SVM achieved Accuracy of 98%, F1 Score ~0.99, and ROC-AUC of 0.9997**, which shows excellent class separation and balance between precision and recall.
- **Random Forest** also gave very strong results with **Accuracy is 98%, F1 Score is 0.98–0.99, and ROC-AUC >0.99**, making it highly reliable.
- I chose **SVM as my final model**, because it consistently gave the **highest ROC-AUC** and slightly better F1-score, meaning it balances false positives and false negatives extremely well.
- However, I also considered **Random Forest** since it is more interpretable and provides feature importance, which is useful in explaining model decisions.
- In conclusion, **SVM is my primary choice for deployment** due to its **near-perfect class separation**, while **Random Forest can be used as a backup/explainable model** if interpretability is a priority.

THANK YOU